Vakint

Single-scale vacuum-integral matching and evaluation

---

1 Overview

Vakint matches single-scale vacuum integrals to a library of known topologies, canonicalizes their momentum routing, reduces tensor numerators, and evaluates them analytically or numerically. Expressions are represented with Symbolica and contain a numerator multiplied by a `topo(...)` structure built from propagators.

> **Separate matching from evaluation**
> Canonical topology matching is useful on its own and is much cheaper than a complete evaluation. Tensor reduction and analytic backends invoke FORM; numerical sector decomposition invokes pySecDec and FORM. Choose the narrowest operation and evaluation order needed for the task.

1.1 Workflow

A complete calculation proceeds through explicit stages:

- create one `Vakint` engine and reuse it, because topology setup has a cost;
- parse or construct a Symbolica expression in Vakint's namespace;
- canonicalize the topology and momentum routing;
- tensor-reduce the numerator when required;
- evaluate the integral with the first supported backend in the configured order;
- substitute numerical parameters and external momenta when a numerical result is required.

`VakintExpression` is the structured sum of numerator/topology terms. A recognized topology can be rendered in canonical long or short form. If unknown integrals are allowed, matching may preserve them, but evaluation still needs a backend that supports the resulting topology.

1.2 External tools and reproducibility

> **Construction validates the selected backends**
> The default evaluation order includes AlphaLoop, MATAD, FMFT, and pySecDec. Settings validation therefore probes FORM and pySecDec even if a later example only canonicalizes an expression. Vakint requires FORM 4.2.1 or newer and pySecDec 1.6.4 or newer. For matching without evaluation, use an empty evaluation order in Rust; in Python, provide an empty `evaluation_order` when constructing `Vakint`.

Analytic AlphaLoop, MATAD, and FMFT methods depend on FORM. The pySecDec method depends on both FORM and pySecDec and requires numerical masses and, where applicable, external momenta. Record tool versions, normalization convention, epsilon depth, decimal precision, backend order, and temporary-output reuse policy with any result intended to be reproduced.

1.3 Diagnostics and platform notes

Matching with an empty evaluation order needs neither FORM nor pySecDec. For backend evaluation, check the configured executable paths first. Verbose logging and retained temporary files are useful when an external tool fails:

```
VAKINT_NO_CLEAN_TMP_DIR=T RUST_LOG=DEBUG cargo run
```

On macOS, if a GNU-compiler link fails with missing `__emul...` symbols, retry the build with `EXTRA_MACOS_LIBS_FOR_GNU_GCC=T` set.

Vakint is used by GammaLoop for vacuum-integral work, but it owns its topology and evaluation conventions independently. The Vakint README contains additional Rust examples; use the API and topology references in this manual for the selected version.

2 Tutorial

This tutorial parses and canonicalizes one vacuum integral in Rust. It deliberately disables all evaluation backends, so the first success exercises Vakint's topology library and momentum routing without requiring FORM, MATAD, FMFT, or pySecDec.

2.1 Prerequisites

Use Rust 1.85 or newer. Clone the workspace, create a sibling binary project, and add the current Vakint crate by path:

```
git clone https://github.com/alphal00p/gammaloop.git
cargo new vakint-first-match
cd vakint-first-match
cargo add vakint --path ../gammaloop/crates/vakint
```

Vakint uses Symbolica; ensure your use satisfies its license terms. External integration tools are unnecessary for the matching-only settings below. The example targets the current repository API; check out a release tag before adding the path dependency when you need a fixed version.

2.2 Match a one-loop topology

Replace `src/main.rs` with:

```
use vakint::{
    vakint_parse, EvaluationOrder, Vakint, VakintExpression, VakintSettings,
};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let settings = VakintSettings {
        allow_unknown_integrals: false,
        evaluation_order: EvaluationOrder::empty(),
        ..VakintSettings::default()
    };
    let vakint = Vakint::new()?;
    vakint.validate_settings(&settings)?;

    let input = vakint_parse!(
        "topo(prop(1,edge(1,1),k(1),muvsq,1))"
    )?;
    let canonical = vakint.to_canonical(
        &settings,
        input.as_view(),
        true,
    )?;

    println!("input: {}", VakintExpression::try_from(input)?);
    println!("canonical: {}", VakintExpression::try_from(canonical)?);
```

```
    Ok(())
}
```

Run `cargo run`. Success means Vakint recognizes the propagator as a supported one-loop vacuum topology, prints a canonical short form, and exits without probing an external executable. The canonical spelling may reorder identifiers or momentum routing; that normalized result, not the input's labels, is the comparison boundary.

> **An empty evaluation order is intentional**
> `EvaluationOrder::empty()` makes this a topology-matching tutorial. Default settings include analytic and numerical backends, and validating those settings checks their external dependencies even if you only planned to call `to_canonical`.

2.3 Move from matching to evaluation

Once the canonical form is understood, choose one supported backend explicitly, validate its dependencies, then apply the stages in order: tensor reduction, integral evaluation, and numerical substitution. Record the normalization factor, epsilon depth, decimal precision, backend settings, and tool versions with the result. Reuse one `Vakint` engine because topology construction has a cost.

2.4 Troubleshooting and next steps

- An unrecognized-integral error means the propagators, powers, masses, or momentum routing did not match a registered topology and `allow_unknown_integrals` is false. Print the long form and compare it with the supported-topology reference.
- A FORM or pySecDec version error means a nonempty evaluation order slipped into the settings; return to `EvaluationOrder::empty()` for pure canonicalization.
- If two inputs look different after matching, compare their canonical forms rather than their original propagator numbering.
- Continue with the topology-and-evaluation manual before enabling a backend or adding tensor numerators.

3 Topology matching, reduction, and evaluation

Vakint recognizes an integral only after its propagators, masses, powers, and momentum routing can be mapped to a supported topology pattern. The topology reference lists the patterns available in the selected version.

3.1 Parsing and topology matching

A `VakintExpression` is a sum of numerator/topology pairs. Parsing checks the Vakint namespace; matching then searches allowed substitutions and momentum shifts. Unknown-topology handling is a setting: preserving an unknown term is useful for partial simplification, but does not make that term evaluable.

Canonical long form is explicit and suitable for auditing. Canonical short form is convenient for display and library lookup. Both represent the same matched topology only after canonicalization has succeeded.

3.2 Normalization and canonicalization

Normalization conventions cover loop-measure factors, mass scales, epsilon powers, and the requested expansion depth. Set them before comparing results from different backends. Momentum canonicalization may rename loop variables and reorder propagators; compare canonical expressions rather than input spelling when testing equivalence.

> **Matching does not require an evaluation backend**
> Parsing, matching, normalization, and canonical routing work without FORM or pySecDec when the evaluation order is empty. Use this configuration when you only need a canonical topology.

3.3 Tensor reduction

Tensor numerators are reduced to scalar integrals before analytic evaluation where required. Reduction depends on the topology, Lorentz rank, dimension convention, and scalar-product normalization. Tensor reduction requires FORM. When diagnosing a mismatch, keep the FORM input and Vakint temporary directory so that the failing reduction can be inspected.

3.4 Evaluation order and backends

The evaluation order is an ordered fallback policy, not a request to combine backend results. Vakint tries each configured method whose capability matches the canonical topology and stops at the first successful evaluation. AlphaLoop, MATAD, and FMFT are analytic FORM-backed paths. pySecDec is numerical and additionally needs numeric parameters and external momenta where the topology requires them.

For reproducible comparisons record:

- the canonical topology and normalization convention;
- epsilon expansion depth and decimal precision;
- the ordered backend list and backend-specific settings;
- FORM, pySecDec, MATAD, and FMFT versions where applicable;
- parameter substitutions and any retained temporary-output directory.

> **Python is an embedded community module**
> `symbolica.community.vakint` is registered into a Symbolica installation; it is not a standalone PyPI package. Constructing its `Vakint` class validates the configured backends. Pass an empty evaluation order for pure matching work on machines without FORM/pySecDec.

See the supported-topology and external-dependency reference for the patterns and minimum tool versions available here. Their Rust definitions begin in Vakint's topology module.

4 Rust and Python APIs

> **API reference**
> The reference for this version lists public items, signatures, fields, defaults, feature requirements, and links to their source definitions.
>
> Rust packages: `vakint`
> Python modules: `symbolica.community.vakint`

4.1 Rust package

The main Rust types make the workflow state explicit:

- `Vakint` owns the topology library and matching/evaluation operations;
- `VakintSettings` owns symbols, precision, normalization, tool paths, validation, and backend order;
- `VakintExpression` and `VakintTerm` separate numerators from topology atoms;
- `EvaluationOrder` and `EvaluationMethod` select AlphaLoop, MATAD, FMFT, or pySecDec;
- `NumericalEvaluationResult` stores Laurent coefficients in the dimensional regulator.

For a dependency-free canonicalization setup, select an empty backend order before validating:

```
use vakint::{EvaluationOrder, Vakint, VakintSettings};

let engine = Vakint::new()?;
let settings = VakintSettings {
    evaluation_order: EvaluationOrder::empty(),
```

```
    ..VakintSettings::default()
};
engine.validate_settings(&settings)?;
```

`vakint_parse!` and the related helpers parse Symbolica expressions in Vakint's namespace. Handle parse failures before passing a user-supplied expression to matching or evaluation.

The curated Rust API presents the supported entry points before the full Rustdoc, while the generated topology reference records the runtime topology library and supported external-tool versions for this release.

4.2 Python community module

> **Python availability**
> Python users import `symbolica.community.vakint`; Vakint is not distributed as a standalone Python package. Check that the installed Symbolica distribution includes the Vakint community module before relying on this interface.

Verify availability with `python -c "import symbolica.community.vakint"`. Custom Symbolica builds can add Vakint to the symbolica-community assembly, enable the `symbolica_community_module` feature, and register `VakintWrapper` while building the extension.

The structured Python reference covers the four exported classes: `Vakint`, `VakintExpression`, `VakintEvaluationMethod`, and `VakintNumericalResult`. This example constructs a matching-only engine without probing external backends:

```
from symbolica.community.vakint import Vakint

vakint = Vakint(evaluation_order=[])
# Supply a Symbolica Expression in the Vakint namespace to `to_canonical`.
canonical = vakint.to_canonical(expr, short_form=True)
```

`Vakint` exposes canonicalization, tensor reduction, integral evaluation, complete evaluation, and numerical conversion. `VakintEvaluationMethod` constructs configured backend choices. `VakintNumericalResult` converts Laurent coefficients to Python tuples and compares results with thresholds and optional errors.

Construct one engine and reuse it. When enabling pySecDec, provide numerical masses and external momenta where the topology requires them. Reuse existing pySecDec output only while debugging, and remove it before evaluating changed inputs.

Source starting points are the Rust API and the Python module.

5 Versions and compatibility

> **Before upgrading**
> Pin the Vakint version together with FORM, pySecDec, MATAD, and FMFT versions used for a calculation. Changes to topology matching, normalization, or a backend can change a result even when the high-level method call remains the same.

The Rust crate and the Python interface included with Symbolica can be released on different schedules. Record both versions when sharing a calculation or reporting a problem.

There is not yet a standalone Vakint changelog. When evaluating an upgrade, check the selected release's API reference and Vakint README, then compare the following areas:

- supported topologies and canonical momentum routing;
- normalization conventions and epsilon-expansion defaults;

- tensor reduction and analytic backend behavior;
- pySecDec integration and numerical precision;
- Rust and Python method signatures and feature requirements.

API reference

The packages available in this version are listed below. Follow an item link for its complete language reference.

`vakint`

Vakint's supported topology and evaluation API. Generated topology and backend tables are *validated* against runtime sources.

`Vakint expression parser`

```
Parses a Symbolica expression using Vakint's namespace by default or a caller-
supplied namespace.
```

```
Parses a Symbolica expression using Vakint's namespace by default or a caller-
supplied namespace.
```

```
#[macro_export] macro_rules! vakint_parse
{
    ($s: expr) =>
    {{ symbolica::try_parse!($s, default_namespace = $crate::NAMESPACE) }};
    ($s: expr, $ns: expr) =>
    {{ symbolica::try_parse!($s, default_namespace = $ns) }};
}
```

Supported usage

```
let expression = vakint::vakint_parse!("x").expect("valid Vakint expression"); let
explicit = vakint::vakint_parse!("x", vakint::NAMESPACE).expect("valid explicit
namespace");
```

Exhaustive reference · Source

`Vakint symbol parser`

```
Resolves a symbol name in Vakint's namespace while preserving an explicit namespace.
```

```
Resolves a symbol name in Vakint's namespace while preserving an explicit namespace.
```

```
#[macro_export] macro_rules! vakint_symbol
{
    ($s: expr) =>
    {{
            if
            format!("{}",
            $s).starts_with(format!("{}::", $crate::NAMESPACE).as_str())
            {
                match symbolica::try_parse!($s)
                {
                    Ok::<_, String>(a) => match a
                    {
                        symbolica::atom::Atom::Var(sym) => sym.get_symbol(), _ =>
                        panic!("Parsed atom is not a symbol: {}.", $s),
                    }, Err(e) =>
                    panic!("Could not parse symbol from string: {}, Error: {}",
                    $s, e),
                }
            } else
            {
                match
                symbolica::try_parse!(format!("{}", $s), default_namespace =
                $crate::NAMESPACE)
```

```
                {
                    Ok::<_, String>(a) => match a
                    {
                        symbolica::atom::Atom::Var(sym) => sym.get_symbol(), _ =>
                        panic!("Parsed atom is not a symbol: {}.", $s),
                    }, Err(e) =>
                    panic!("Could not parse symbol from string: {}, Error: {}",
                    $s, e),
                }
            }
        }};
}
```

Supported usage

```
let symbol = vakint::vakint_symbol!("mass");
```

Exhaustive reference · Source

`Vakint engine`

Owns supported topology matching, canonicalization, reduction, and evaluation operations.

Owns supported topology matching, canonicalization, reduction, and evaluation operations.

```
#[derive(Debug, Clone)] pub struct Vakint { pub topologies : Topologies, }
```

Supported usage

```
let engine = vakint::Vakint::new()?;
```

`topologies`

Kind: `Field`

```
Topologies
```

Exhaustive reference · Source

`Vakint settings`

Controls symbols, precision, normalization, external tools, and backend evaluation order.

Controls symbols, precision, normalization, external tools, and backend evaluation order.

```
#[derive(Debug, Clone)] pub struct VakintSettings
{
    #[allow(unused)] pub epsilon_symbol : String, pub mu_r_sq_symbol : String,
    pub form_exe_path : String, pub python_exe_path : String, pub
    verify_numerator_identification : bool, pub integral_normalization_factor
    : LoopNormalizationFactor, pub run_time_decimal_precision : u32, pub
    allow_unknown_integrals : bool, pub clean_tmp_dir : bool, pub
    evaluation_order : EvaluationOrder, pub
    number_of_terms_in_epsilon_expansion : i64, pub
    precision_for_input_float_rationalization :
    InputFloatRationalizationPrecision, pub use_dot_product_notation : bool,
    pub temporary_directory : Option < String > ,
}
```

Supported usage

```
let settings = vakint::VakintSettings::default();
```

epsilon_symbol
Kind: Field

String

mu_r_sq_symbol
Kind: Field

String

form_exe_path
Kind: Field

String

python_exe_path
Kind: Field

String

verify_numerator_identification
Kind: Field

bool

integral_normalization_factor
Kind: Field

LoopNormalizationFactor

run_time_decimal_precision
Kind: Field

u32

allow_unknown_integrals
Kind: Field

bool

clean_tmp_dir
Kind: Field

bool

evaluation_order
Kind: Field

EvaluationOrder

number_of_terms_in_epsilon_expansion
Kind: Field

i64

precision_for_input_float_rationalization
Kind: Field

InputFloatRationalizationPrecision

use_dot_product_notation

Kind: Field

bool

temporary_directory

Kind: Field

Option < String >

Exhaustive reference · Source

Vakint expression

Stores a sum of matched vacuum-integral terms with separate numerators.

Stores a sum of matched vacuum-integral terms with separate numerators.

#[derive(Debug, Clone, PartialEq, Eq)] pub struct
VakintExpression(pub Vec < VakintTerm >);

Supported usage

```
use vakint::VakintExpression;
```

0

Kind: Field

Vec < VakintTerm >

Exhaustive reference · Source

Evaluation order

Defines the ordered analytic and numerical backends available to an evaluation.

Defines the ordered analytic and numerical backends available to an evaluation.

#[derive(Debug, Clone)] pub struct
EvaluationOrder(pub Vec < EvaluationMethod >);

Supported usage

```
let order = vakint::EvaluationOrder::empty();
```

0

Kind: Field

Vec < EvaluationMethod >

Exhaustive reference · Source

Canonicalize an expression

Matches and rewrites each integral into Vakint's canonical topology representation.

Matches and rewrites each integral into Vakint's canonical topology representation.

fn
canonicalize(& mut self, settings : & VakintSettings, topologies : &
Topologies, short_form : bool,) -> Result < (), VakintError >

Parameter	Type
self	& mut self

Parameter	Type
settings	& VakintSettings
topologies	& Topologies
short_form	bool

Returns: `Result < (), VakintError >`

Supported usage

```
use vakint::VakintExpression;
```

Exhaustive reference · Source

`Tensor-reduce an expression`

Reduces tensor numerators to scalar-integral combinations.

Reduces tensor numerators to scalar-integral combinations.

```
fn tensor_reduce(& mut self, vakint : & Vakint, settings : & VakintSettings,)
-> Result < (), VakintError >
```

Parameter	Type
self	& mut self
vakint	& Vakint
settings	& VakintSettings

Returns: `Result < (), VakintError >`

Supported usage

```
use vakint::VakintExpression;
```

Exhaustive reference · Source

`Evaluate an expression`

Evaluates canonical scalar integrals using the configured ordered backend policy.

Evaluates canonical scalar integrals using the configured ordered backend policy.

```
fn
evaluate_integral(& mut self, vakint : & Vakint, settings : & VakintSettings,)
-> Result < (), VakintError >
```

Parameter	Type
self	& mut self
vakint	& Vakint
settings	& VakintSettings

Returns: `Result < (), VakintError >`

Supported usage

```
use vakint::VakintExpression;
```

Exhaustive reference · Source

`Numerical Laurent result`

Stores numerical Laurent coefficients by power of the dimensional regulator.

Stores numerical Laurent coefficients by power of the dimensional regulator.

#[derive(Debug, Clone, Default)] pub struct
NumericalEvaluationResult(pub Vec < (i64, Complex < Float >) >);

Supported usage

```
use vakint::NumericalEvaluationResult;
```

0

Kind: `Field`

Vec < (i64, Complex < Float >) >

Exhaustive reference · Source

`vakint-community`

Exports registered by vakint.

`Vakint`

Base class of vakint engine, from which all vakint functions are initiated.

Base class of vakint engine, from which all vakint functions are initiated.
It is best to create a single instance of this class and reuse it for multiple evaluations,
as the setup of the instance can be time consuming since it involves the processing of all known topologies.

```
class Vakint:
```

Requires features: `symbolica_community_module, python_stubgen`

Inspect the registered export

```
from symbolica.community.vakint import Vakint
help(Vakint)
```

`__new__`

Kind: `AssociatedFunction`

```
def __new__(cls, run_time_decimal_precision: typing.Optional[builtins.int] = None,
evaluation_order: typing.Optional[typing.Sequence[VakintEvaluationMethod]] = None,
epsilon_symbol: typing.Optional[Expression] = None, mu_r_sq_symbol:
typing.Optional[Expression] = None, form_exe_path: typing.Optional[builtins.str] =
None, python_exe_path: typing.Optional[builtins.str] = None,
verify_numerator_identification: typing.Optional[builtins.bool] = None,
integral_normalization_factor: typing.Optional[builtins.str] = None,
allow_unknown_integrals: typing.Optional[builtins.bool] = None, clean_tmp_dir:
typing.Optional[builtins.bool] = None, number_of_terms_in_epsilon_expansion:
typing.Optional[builtins.int] = None, use_dot_product_notation:
typing.Optional[builtins.bool] = None, temporary_directory:
typing.Optional[builtins.str] = None) -> Vakint:
```

Create a new Vakint instance, specifying details of the evaluation stack. Note that the same instance can be recycled across multiple evaluations.
Note that the creation of a Vakint instance involves the processing and creation of the library of all known topologies, which can be time consuming.

Examples
```python
vakint = Vakint(
integral_normalization_factor="MSbar",
```

```
mu_r_sq_symbol=S("mursq"),
# If you select 5 terms, then MATAD will be used, but for 4 and fewer, alphaLoop is
will be used as
# it is first in the evaluation_order supplied.
number_of_terms_in_epsilon_expansion=4,
evaluation_order=[
VakintEvaluationMethod.new_alphaloop_method(),
VakintEvaluationMethod.new_matad_method(),
VakintEvaluationMethod.new_fmft_method(),
VakintEvaluationMethod.new_pysecdec_method(
min_n_evals=10_000,
max_n_evals=1000_000,
numerical_masses=masses,
numerical_external_momenta=external_momenta
),
],
form_exe_path="form",
python_exe_path="python3",
)
```

Parameters
----

run_time_decimal_precision : Optional[int]
The decimal precision to be used during the evaluation. Default is 17.
evaluation_order : Optional[Sequence[VakintEvaluationMethod]]
A list of `VakintEvaluationMethod` instances specifying the order in which evaluation
methods are to be applied. Default is all available methods in a sensible order.
epsilon_symbol : Optional[Expression]
The symbol to be used for the dimensional regularisation parameter epsilon. Default
is "ε".
mu_r_sq_symbol : Optional[Expression]
The symbol to be used for the renormalisation scale squared. Default is "mursq".
form_exe_path : Optional[str]
The path to the FORM executable. Default is "form".
python_exe_path : Optional[str]
The path to the Python executable. Default is "python3".
verify_numerator_identification : Optional[bool]
Whether to verify the identification of numerator structures. Default is True.
integral_normalization_factor : Optional[str]
The normalization factor to be used for integrals. Can be "MSbar", "pySecDec",
"FMFTandMATAD" or a custom string. Default is "MSbar".
allow_unknown_integrals : Optional[bool]
Whether to allow unknown integrals to be processed. Default is True.
clean_tmp_dir : Optional[bool]
Whether to clean the temporary directory after evaluation. Default is True, unless
the environment variable VAKINT_NO_CLEAN_TMP_DIR is set.
number_of_terms_in_epsilon_expansion : Optional[int]
The number of terms in the epsilon expansion to be computed. Default is 4.
use_dot_product_notation : Optional[bool]
Whether to use dot product notation for scalar products. Default is False.
temporary_directory : Optional[str]
The path to the temporary directory to be used. Default is None, in which case a
system temporary directory will be used.

run_time_decimal_precision

Kind: Parameter

```
run_time_decimal_precision: typing.Optional[builtins.int] = None
```

Default: None

The decimal precision to be used during the evaluation. Default is 17.

evaluation_order

Kind: Parameter

```
evaluation_order: typing.Optional[typing.Sequence[VakintEvaluationMethod]] = None
```

Default: None

A list of `VakintEvaluationMethod` instances specifying the order in which evaluation methods are to be applied. Default is all available methods in a sensible order.

epsilon_symbol

Kind: Parameter

```
epsilon_symbol: typing.Optional[Expression] = None
```

Default: None

The symbol to be used for the dimensional regularisation parameter epsilon. Default is "ε".

mu_r_sq_symbol

Kind: Parameter

```
mu_r_sq_symbol: typing.Optional[Expression] = None
```

Default: None

The symbol to be used for the renormalisation scale squared. Default is "mursq".

form_exe_path

Kind: Parameter

```
form_exe_path: typing.Optional[builtins.str] = None
```

Default: None

The path to the FORM executable. Default is "form".

python_exe_path

Kind: Parameter

```
python_exe_path: typing.Optional[builtins.str] = None
```

Default: None

The path to the Python executable. Default is "python3".

verify_numerator_identification

Kind: Parameter

```
verify_numerator_identification: typing.Optional[builtins.bool] = None
```

Default: None

Whether to verify the identification of numerator structures. Default is True.

integral_normalization_factor

Kind: Parameter

```
integral_normalization_factor: typing.Optional[builtins.str] = None
```

Default: None

The normalization factor to be used for integrals. Can be "MSbar", "pySecDec", "FMFTandMATAD" or a custom string. Default is "MSbar".

allow_unknown_integrals

Kind: Parameter

```
allow_unknown_integrals: typing.Optional[builtins.bool] = None
```

Default: None

Whether to allow unknown integrals to be processed. Default is True.

clean_tmp_dir

Kind: Parameter

```
clean_tmp_dir: typing.Optional[builtins.bool] = None
```

Default: None

Whether to clean the temporary directory after evaluation. Default is True, unless the environment variable VAKINT_NO_CLEAN_TMP_DIR is set.

number_of_terms_in_epsilon_expansion

Kind: Parameter

```
number_of_terms_in_epsilon_expansion: typing.Optional[builtins.int] = None
```

Default: None

The number of terms in the epsilon expansion to be computed. Default is 4.

use_dot_product_notation

Kind: Parameter

```
use_dot_product_notation: typing.Optional[builtins.bool] = None
```

Default: None

Whether to use dot product notation for scalar products. Default is False.

temporary_directory

Kind: Parameter

```
temporary_directory: typing.Optional[builtins.str] = None
```

Default: None

The path to the temporary directory to be used. Default is None, in which case a system temporary directory will be used.

numerical_result_from_expression

Kind: Method

```
def numerical_result_from_expression(self, expr: Expression) ->
VakintNumericalResult:
```

Convert a Symbolica expression to a vakint numerical result, interpreting the
expression as a Laurent series in the dimensional regularisation parameter epsilon.

Examples
```python
res = vakint.numerical_result_from_expression(E("vakint::ε^-2 + 1 +
0.12*vakint::ε^-1"))
str(res)
# ε^-2 : (1.000000000000000+0i)
# ε^-1 : (1.200000000000000e-1+0i)
# ε^ 0 : (1.000000000000000+0i)
```

Parameters
----

expr : Expression
A Symbolica expression representing a Laurent series in the dimensional
regularisation parameter epsilon specified in the vakint engine.

expr

Kind: Parameter

expr: Expression

A Symbolica expression representing a Laurent series in the dimensional
regularisation parameter epsilon specified in the vakint engine.

numerical_evaluation

Kind: Method

def numerical_evaluation(self, evaluated_integral: typing.Any, params:
typing.Mapping[builtins.str, builtins.float], externals:
typing.Optional[typing.Mapping[builtins.int, tuple[builtins.float, builtins.float,
builtins.float, builtins.float]]] = None) -> tuple[VakintNumericalResult,
typing.Optional[VakintNumericalResult]]:

Perform a numerical evaluation of an integral parameterically evaluated by Vakint,
given numerical values for all parameters and optionally for external momenta.

Examples
```python
evaluated_integral =
vakint.evaluate_integral(E("(k(1,11)*k(1,11)+p(1,12)*p(2,12))*topo(prop(1,edge(1,1),k(1),muvsq,1))"
default_namespace="vakint"))
numerical_result, numerical_error = vakint.numerical_evaluation(
evaluated_integral,
{ "muvsq": 2., "mursq": 3.},
{
1: (0.1, 0.2, 0.3, 0.4),
2: (0.5, 0.6, 0.7, 0.8)
}
)
str(numerical_result)
# ε^-1 : (0+27.63489232305020i)
# ε^ 0 : (0+-62.73919274007806i)
# ε^+1 : (0+107.7642844179578i)
# ε^+2 : (0+-138.7269737122023i)
```

```

Parameters
----------

evaluated_integral : Expression
A Symbolica expression representing an integral that has been evaluated
parameterically by Vakint
params : Dict[str, float]
A dictionary mapping parameter names to their numerical values.
externals : Optional[Dict[int, Tuple[float, float, float, float]]]
An optional dictionary mapping external momentum indices to their numerical 4-vector
values.

**evaluated_integral**

**Kind:** Parameter

evaluated_integral: typing.Any

A Symbolica expression representing an integral that has been evaluated
parameterically by Vakint

**params**

**Kind:** Parameter

params: typing.Mapping[builtins.str, builtins.float]

A dictionary mapping parameter names to their numerical values.

**externals**

**Kind:** Parameter

externals: typing.Optional[typing.Mapping[builtins.int, tuple[builtins.float,
builtins.float, builtins.float, builtins.float]]] = None

**Default:** None

An optional dictionary mapping external momentum indices to their numerical 4-vector
values.

**numerical_result_to_expression**

**Kind:** Method

def numerical_result_to_expression(self, result: VakintNumericalResult) ->
Expression:

Convert a vakint numerical result back to a Symbolica expression representing a
Laurent series in the dimensional regularisation parameter epsilon.

## Examples
```python
evaluated_integral =
vakint.evaluate_integral(E("(k(1,11)*k(1,11)+p(1,12)*p(2,12))*topo(prop(1,edge(1,1),k(1),muvsq,1))"
default_namespace="vakint"))
numerical_result, numerical_error = vakint.numerical_evaluation(
evaluated_integral,
{ "muvsq": 2., "mursq": 3.},
{
1: (0.1, 0.2, 0.3, 0.4),
2: (0.5, 0.6, 0.7, 0.8)
```
```

```
}
)
vakint.numerical_result_to_expression(numerical_result)
#
107.7642844179578𝑖*ε+27.63489232305020𝑖*ε^-1+-138.7269737122023𝑖*ε^2+-62.73919274007806𝑖
```

result

Kind: Parameter

```
result: VakintNumericalResult
```

to_canonical

Kind: Method

```
def to_canonical(self, integral_expression: Expression, short_form:
typing.Optional[builtins.bool] = None) -> Expression:
```

```
Convert a vakint expression to its canonical form, optionally using a short form for
the topology representation.

## Examples
```python
integral_expr =
E("(k(99,11)*k(99,11)+p(1,12)*p(2,12))*topo(prop(18,edge(7,7),k(99),muvsq,1))",
default_namespace="vakint")
vakint.to_canonical(integral_expr)
# (k(1,11)^2+p(1,12)*p(2,12))*topo(prop(1,edge(1,1),k(1),muvsq,1))
vakint.to_canonical(integral_expr,short_form=True)
# (k(1,11)^2+p(1,12)*p(2,12))*topo(I1L(muvsq,1))
```

Parameters
----

integral_expression : Expression
A Symbolica expression representing a vakint integral.
short_form : Optional[bool]
Whether to use the short form for the topology representation. Default is False.
```

integral_expression

Kind: Parameter

```
integral_expression: Expression
```

```
A Symbolica expression representing a vakint integral.
```

short_form

Kind: Parameter

```
short_form: typing.Optional[builtins.bool] = None
```

Default: None

```
Whether to use the short form for the topology representation. Default is False.
```

tensor_reduce

Kind: Method

```
def tensor_reduce(self, integral_expression: Expression) -> Expression:
```

Convert a vakint expression to a form where tensor integrals are reduced to scalar integrals.

Examples
```python
integral_expr =
E("(k(1,11)*k(1,11)+p(1,12)*k(1,12)+k(1,101)*k(1,102))*topo(prop(1,edge(1,1),k(1),muvsq,1))",
default_namespace="vakint")

vakint.tensor_reduce(integral_expr)
# (k(1,1)^2-(2*ε-4)^-1*k(1,1)^2*g(101,102))*topo(prop(1,edge(1,1),k(1),muvsq,1))
```

Parameters
----
integral_expression : Expression
A Symbolica expression representing a vakint integral.

integral_expression

Kind: Parameter

integral_expression: Expression

A Symbolica expression representing a vakint integral.

evaluate_integral

Kind: Method

def evaluate_integral(self, integral_expression: Expression) -> Expression:

Perform the parametric evaluation of *only the integral* appearing in the Symbolica expression given in input representing a vakint integral.
The numerator is left unchanged.

Examples
```python
integral_expr =
E("(k(1,11)*k(1,11)+p(1,12)*k(1,12)+k(1,101)*k(1,102))*topo(prop(1,edge(1,1),k(1),muvsq,1))",
default_namespace="vakint")

vakint.evaluate_integral(integral_expr)
# ε*(-((-π^2*i*log(π)+π^2*i*log(1/4*π^-1*mursq))*(muvsq^2+muvsq*k(1,1)*p(1,1)+[...]
```

Parameters
----

integral_expression : Expression
A Symbolica expression representing a vakint integral.

integral_expression

Kind: Parameter

integral_expression: Expression

A Symbolica expression representing a vakint integral.

evaluate

Kind: Method

```
def evaluate(self, integral_expression: Expression) -> Expression:

Perform the complete parametric evaluation of the vaking integral represented by the
Symbolica expression given in input.
Note that the tensor reduction will be automatically performed on the input given.

## Examples
```python
integral_expr =
E("(k(1,11)*k(1,11)+p(1,12)*k(1,12)+k(1,101)*k(1,102))*topo(prop(1,edge(1,1),k(1),muvsq,1))",
default_namespace="vakint")

vakint.evaluate(integral_expr)
#
ε*((muvsq^2+1/4*muvsq^2*g(101,102))*(1/2*π^2*i*log(π)^2+1/2*π^2*i*log(1/4*π^-1*mursq)^2-
[...]
```

Parameters
----

integral_expression : Expression
A Symbolica expression representing a vakint integral.
```

integral_expression

Kind: `Parameter`

```
integral_expression: Expression

A Symbolica expression representing a vakint integral.
```

Exhaustive reference · Source

VakintEvaluationMethod

```
Class representing a vakint evaluation method, which can be used to specify the
evaluation order in a Vakint instance.

Class representing a vakint evaluation method, which can be used to specify the
evaluation order in a Vakint instance.

class VakintEvaluationMethod:
```

Requires features: `symbolica_community_module, python_stubgen`

Inspect the registered export

```
from symbolica.community.vakint import VakintEvaluationMethod
help(VakintEvaluationMethod)
```

__str__

Kind: `Method`

```
def __str__(self) -> builtins.str:

String representation of the evaluation method.
```

new_alphaloop_method

Kind: `AssociatedFunction`

```
def new_alphaloop_method(cls) -> VakintEvaluationMethod:

Create a new VakintEvaluationMethod instance representing the AlphaLoop method.
This method does not take any parameters.
```

Examples
```python
alphaloop_method = VakintEvaluationMethod.new_alphaloop_method()
```

new_matad_method

Kind: AssociatedFunction

```
def new_matad_method(cls, expand_masters: typing.Optional[builtins.bool] = None,
susbstitute_masters: typing.Optional[builtins.bool] = None, substitute_hpls:
typing.Optional[builtins.bool] = None, direct_numerical_substition:
typing.Optional[builtins.bool] = None) -> VakintEvaluationMethod:
```

Create a new VakintEvaluationMethod instance representing the MATAD method.

Examples
```python
matad_method = VakintEvaluationMethod.new_matad_method(
expand_masters=True,
susbstitute_masters=True,
substitute_hpls=True,
direct_numerical_substition=True
)
```

Parameters
----

expand_masters : Optional[bool]
Whether to expand master integrals. Default is True.
susbstitute_masters : Optional[bool]
Whether to substitute master integrals. Default is True.
substitute_hpls : Optional[bool]
Whether to substitute harmonic polylogarithms. Default is True.
direct_numerical_substition : Optional[bool]
Whether to perform direct numerical substitution. Default is True.

expand_masters

Kind: Parameter

```
expand_masters: typing.Optional[builtins.bool] = None
```

Default: None

Whether to expand master integrals. Default is True.

susbstitute_masters

Kind: Parameter

```
susbstitute_masters: typing.Optional[builtins.bool] = None
```

Default: None

Whether to substitute master integrals. Default is True.

substitute_hpls

Kind: Parameter

```
substitute_hpls: typing.Optional[builtins.bool] = None
```

Default: None

Whether to substitute harmonic polylogarithms. Default is True.

direct_numerical_substition

Kind: Parameter

direct_numerical_substition: typing.Optional[builtins.bool] = None

Default: None

Whether to perform direct numerical substitution. Default is True.

new_fmft_method

Kind: AssociatedFunction

def new_fmft_method(cls, expand_masters: typing.Optional[builtins.bool] = None, susbstitute_masters: typing.Optional[builtins.bool] = None) -> VakintEvaluationMethod:

Create a new VakintEvaluationMethod instance representing the FMFT method.

Examples
```python
fmft_method = VakintEvaluationMethod.new_fmft_method(
expand_masters=True,
susbstitute_masters=True
)
```

Parameters
----

expand_masters : Optional[bool]
Whether to expand master integrals. Default is True.
susbstitute_masters : Optional[bool]
Whether to substitute master integrals. Default is True.

expand_masters

Kind: Parameter

expand_masters: typing.Optional[builtins.bool] = None

Default: None

Whether to expand master integrals. Default is True.

susbstitute_masters

Kind: Parameter

susbstitute_masters: typing.Optional[builtins.bool] = None

Default: None

Whether to substitute master integrals. Default is True.

new_pysecdec_method

Kind: AssociatedFunction

def new_pysecdec_method(cls, quiet: typing.Optional[builtins.bool] = None, relative_precision: typing.Optional[builtins.float] = None, min_n_evals:

typing.Optional[builtins.int] = None, max_n_evals: typing.Optional[builtins.int] = None, reuse_existing_output: typing.Optional[builtins.str] = None, numerical_masses: typing.Optional[typing.Mapping[builtins.str, builtins.float]] = None, numerical_external_momenta: typing.Optional[typing.Mapping[builtins.int, tuple[builtins.float, builtins.float, builtins.float, builtins.float]]] = None) -> VakintEvaluationMethod:

Create a new VakintEvaluationMethod instance representing the numerical pySecDec method.

Examples
```python
pysecdec_method = VakintEvaluationMethod.new_pysecdec_method(
quiet=True,
relative_precision=1e-7,
min_n_evals=10_000,
max_n_evals=1_000_000_000_000,
reuse_existing_output=None,
numerical_masses={"muvsq": 1.0},
numerical_external_momenta={1: (1.0, 0.0, 0.0, 0.0), 2: (0.0, 1.0, 0.0, 0.0)}
)
```

Note that for because pySecDec can only do numerical evaluations, the preset values of the masses and external momenta must be provided here.

Parameters
----

quiet : Optional[bool]
Whether to suppress output from pySecDec. Default is True.
relative_precision : Optional[float]
The relative precision to be achieved in the numerical integration. Default is 1e-7.
min_n_evals : Optional[int]
The minimum number of evaluations to be performed in the numerical integration. Default is 10,000.
max_n_evals : Optional[int]
The maximum number of evaluations to be performed in the numerical integration. Default is 1,000,000,000,000.
reuse_existing_output : Optional[str]
Path to existing pySecDec output to reuse. Default is None.
numerical_masses : Optional[Dict[str, float]]
A dictionary mapping mass parameter names to their numerical values. Default is an empty dictionary.
numerical_external_momenta : Optional[Dict[int, Tuple[float, float, float, float]]]
A dictionary mapping external momentum indices to their numerical 4-vector values. Default is an empty dictionary.

quiet

Kind: Parameter

quiet: typing.Optional[builtins.bool] = None

Default: None

Whether to suppress output from pySecDec. Default is True.

relative_precision

Kind: Parameter

```
relative_precision: typing.Optional[builtins.float] = None
```

Default: None

The relative precision to be achieved in the numerical integration. Default is 1e-7.

min_n_evals

Kind: Parameter

```
min_n_evals: typing.Optional[builtins.int] = None
```

Default: None

The minimum number of evaluations to be performed in the numerical integration. Default is 10,000.

max_n_evals

Kind: Parameter

```
max_n_evals: typing.Optional[builtins.int] = None
```

Default: None

The maximum number of evaluations to be performed in the numerical integration. Default is 1,000,000,000,000.

reuse_existing_output

Kind: Parameter

```
reuse_existing_output: typing.Optional[builtins.str] = None
```

Default: None

Path to existing pySecDec output to reuse. Default is None.

numerical_masses

Kind: Parameter

```
numerical_masses: typing.Optional[typing.Mapping[builtins.str, builtins.float]] =
None
```

Default: None

A dictionary mapping mass parameter names to their numerical values. Default is an empty dictionary.

numerical_external_momenta

Kind: Parameter

```
numerical_external_momenta: typing.Optional[typing.Mapping[builtins.int,
tuple[builtins.float, builtins.float, builtins.float, builtins.float]]] = None
```

Default: None

A dictionary mapping external momentum indices to their numerical 4-vector values. Default is an empty dictionary.

VakintExpression

A Vakint integral split into its numerator and normalized topology structure.

A Vakint integral split into its numerator and normalized topology structure.

Construct this wrapper from a Symbolica expression before applying Vakint operations.

class VakintExpression:

Requires features: symbolica_community_module, python_stubgen

Inspect the registered export

```
from symbolica.community.vakint import VakintExpression
help(VakintExpression)
```

__str__

Kind: Method

def __str__(self) -> builtins.str:

String representation of the VakintExpression.

to_expression

Kind: Method

def to_expression(self) -> Expression:

Convert the VakintExpression back to a Symbolica Expression.

Examples
```python
integral = VakintExpression(E('''
(
k(1,11)*k(1,11)
)*topo(
prop(1,edge(1,1),k(1),muvsq,1)
)''', default_namespace="vakint"))
str(integral)
# (k(1,11)^2) x topo(prop(1,edge(1,1),k(1),muvsq,1))
```

__new__

Kind: AssociatedFunction

def __new__(cls, atom: typing.Any) -> VakintExpression:

Create a new VakintExpression from a Symbolica Expression which will separate numerator and topologies

Examples
```python
integral=E('''
(
k(1,11)*k(2,11)*k(1,22)*k(2,22)
+ p(1,11)*k(3,11)*k(3,22)*p(2,22)
+ p(1,11)*p(2,11)*(k(2,22)+k(1,22))*k(2,22)
)*topo(
prop(1,edge(1,2),k(1),muvsq,1)
* prop(2,edge(2,3),k(2),muvsq,1)
* prop(3,edge(3,1),k(3),muvsq,1)
```

```
* prop(4,edge(1,4),k(3)-k(1),muvsq,1)
* prop(5,edge(2,4),k(1)-k(2),muvsq,1)
* prop(6,edge(3,4),k(2)-k(3),muvsq,1)
)''', default_namespace="vakint")
print(VakintExpression(integral))
#
((k(1,22)+k(2,22))*k(2,22)*p(1,11)*p(2,11)+k(1,11)*k(1,22)*k(2,11)*k(2,22)+k(3,11)*k(3,22)*p(1,11)*
x
topo(prop(1,edge(1,2),k(1),muvsq,1)*prop(2,edge(2,3),k(2),muvsq,1)*prop(3,edge(3,1),k(3),muvsq,1)*p
k(1)+k(3),muvsq,1)*prop(5,edge(2,4),k(1)-k(2),muvsq,1)*prop(6,edge(3,4),k(2)-
k(3),muvsq,1))
```

```
Parameters
----------

atom : Expression
A Symbolica Expression containing a vakint integral, i.e. a sum of terms, each a
product of a numerator and a `vakint::topo(...)` structure.
```

atom

Kind: `Parameter`

```
atom: typing.Any
```

```
A Symbolica Expression containing a vakint integral, i.e. a sum of terms, each a
product of a numerator and a `vakint::topo(...)` structure.
```

Exhaustive reference · Source

VakintNumericalResult

```
Container class storing the result of a numerical evaluation of a vakint expression
as a Laurent series in the dimensional regularisation parameter epsilon.
```


```
Container class storing the result of a numerical evaluation of a vakint expression
as a Laurent series in the dimensional regularisation parameter epsilon.
```


```
class VakintNumericalResult:
```

Requires features: `symbolica_community_module, python_stubgen`

Inspect the registered export

```
from symbolica.community.vakint import VakintNumericalResult
help(VakintNumericalResult)
```

__str__

Kind: `Method`

```
def __str__(self) -> builtins.str:
```

```
String representation of the numerical result.

## Examples
```python
result = VakintNumericalResult([
(-3, (0.0, -11440.53140354612)),
(-2, (0.0,  57169.95521898031)),
(-1, (0.0, -178748.9838377694)),
(0, (0.0,  321554.1122184795)),
])
```
```

```
str(result)
ε^-3 : (0+-11440.5314035461i)
ε^-2 : (0+57169.9552189803i)
ε^-1 : (0+-178748.983837769i)
ε^ 0 : (0+321554.112218480i)
```

to_list

Kind: Method

```
def to_list(self) -> builtins.list[tuple[builtins.int, tuple[builtins.float,
builtins.float]]]:
```

Convert the numerical result to a native Python list of (epsilon exponent, (real, imag)) tuples.

Examples
```python
result = VakintNumericalResult([
(-3, (0.0, -11440.53140354612)),
(-2, (0.0,  57169.95521898031)),
(-1, (0.0, -178748.9838377694)),
(0, (0.0,  321554.1122184795)),
])

result.to_list()
# [(-3, (0.0, -11440.53140354612)), (-2, (0.0, 57169.95521898031)), (-1, (0.0,
-178748.9838377694)), (0, (0.0, 321554.1122184795))]
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, values: typing.Sequence[tuple[builtins.int, tuple[builtins.float,
builtins.float]]]) -> VakintNumericalResult:
```

Create a new instance of VakintNumericalResult from a list of (espilon exponent, (real, imag)) tuples.

Examples
```python
VakintNumericalResult([
(-3, (0.0, -11440.53140354612)),
(-2, (0.0,  57169.95521898031)),
(-1, (0.0, -178748.9838377694)),
(-0, (0.0,  321554.1122184795)),
])
```

```
Parameters
----------

values : List[Tuple[int, Tuple[float, float]]]
A list of tuples, each containing an integer exponent of epsilon and a tuple of two
floats
representing the real and imaginary parts of the coefficient.
```

values

Kind: Parameter

values: typing.Sequence[tuple[builtins.int, tuple[builtins.float, builtins.float]]]

A list of tuples, each containing an integer exponent of epsilon and a tuple of two floats representing the real and imaginary parts of the coefficient.

compare_to

Kind: Method

def compare_to(self, other: VakintNumericalResult, relative_threshold: builtins.float, error: typing.Optional[VakintNumericalResult] = None, max_pull: typing.Optional[builtins.float] = None) -> tuple[builtins.bool, builtins.str]:

Compare this numerical result to another, returning a tuple of (bool, str) where the bool indicates whether the results match within the specified thresholds,
and the str provides details of the comparison.

Examples
```python
result1 = VakintNumericalResult([
(-3, (0.0, -11440.53140354612)),
])
result2 = VakintNumericalResult([
(-3, (0.0, -11440.53140354612)),
(-2, (0.0,  2.0)),
])
result1.compare_to(result2, relative_threshold=1e-5)
# (False, 'imaginary part of ε^-2 coefficient does not match within rel. error
required: 0 != 2.00000000000000 (rel. error = 2.00000000000000)')
```

Parameters
----

other : VakintNumericalResult
The other numerical result to compare to.
relative_threshold : float
The relative threshold for comparison.
error : Optional[VakintNumericalResult]
An optional numerical result representing the error in the evaluation.
max_pull : Optional[float]
The maximum pull for comparison. Default is 3.0.

other

Kind: Parameter

other: VakintNumericalResult

The other numerical result to compare to.

relative_threshold

Kind: Parameter

relative_threshold: builtins.float

The relative threshold for comparison.

error

Kind: `Parameter`

```
error: typing.Optional[VakintNumericalResult] = None
```

Default: `None`

```
An optional numerical result representing the error in the evaluation.
```

max_pull

Kind: `Parameter`

```
max_pull: typing.Optional[builtins.float] = None
```

Default: `None`

```
The maximum pull for comparison. Default is 3.0.
```

Exhaustive reference · Source

Topologies and dependencies

This version supports the topology patterns and external-tool versions below.

External dependencies

Dependency	Minimum version
`FORM`	`4.2.1`
`pySecDec`	`1.6.4`

Supported topology patterns

Name	Loops	Propagator slots
`I1L`	1	1
`I2L`	2	3
`I2L_pinch_3`	2	3
`I3L`	3	6
`I3L_pinch_6`	3	6
`I3L_pinch_1_6`	3	6
`I3L_pinch_3_6`	3	6
`I3L_pinch_1_3_6`	3	6
`I4L_H`	4	9
`I4L_X`	4	9
`I4L_BMW`	4	8
`I4L_FG`	4	8
`I4L_FG_pinch_2`	4	8
`I4L_FG_pinch_3`	4	8
`I4L_FG_pinch_4`	4	8
`I4L_FG_pinch_7`	4	8
`I4L_FG_pinch_8`	4	8
`I4L_FG_pinch_1_8`	4	8
`I4L_FG_pinch_2_4`	4	8
`I4L_FG_pinch_4_7`	4	8
`I4L_FG_pinch_4_8`	4	8
`I4L_FG_pinch_7_8`	4	8
`I4L_FG_pinch_2_4_7`	4	8
`I4L_FG_pinch_3_4_8`	4	8
`I4L_FG_pinch_4_7_8`	4	8
`I4L_FG_pinch_6_7_8`	4	8
`I4L_FG_pinch_4_6_7_8`	4	8

Version information

Save these identifiers with published results and include them in issue reports so that collaborators can reproduce the same software environment.

- **Documentation channel:** `latest`
- **Documentation route:** `/products/vakint/latest/`
- **Source revision:** `7e841ca618e462c31f349352023971274472fdd6`

Package versions and enabled features

- `gammaloop/gammalooprs` — `0.3.4` (`no optional features`)
- `gammaloop/gammaloop-api` — `0.1.0` (`features: cli`)
- `gammaloop/gammaloop` — `0.3.4` (`features: python_api, python_abi, python_stubgen`)
- `linnet/linnet` — `0.17.0` (`features: nodestore-vec, serde, bincode, drawing, symbolica`)
- `linnet/linnet-py` — `0.1.0` (`features: extension-module, abi3-py310`)
- `spenso/spenso` — `0.6.0` (`features: shadowing`)
- `spenso/spenso-macros` — `0.3.2` (`features: shadowing`)
- `spenso/spenso-hep-lib` — `0.1.7` (`no optional features`)
- `spenso/spynso3` — `0.1.2` (`features: python_stubgen`)
- `idenso/idenso` — `0.3.0` (`features: bincode, reference-cases`)
- `idenso/idenso` — `0.3.0` (`features: python, python_stubgen`)
- `vakint/vakint` — `0.1.2` (`no optional features`)
- `vakint/vakint` — `0.1.2` (`features: symbolica_community_module, python_stubgen`)